```
import numpy as np
```

```
def convert_units(value, conversion_type):
    if conversion_type == "m_to_ft":
        return value * 3.28084
    elif conversion_type == "ft_to_m":
        return value / 3.28084
    elif conversion_type == "kg_to_lbs":
        return value * 2.20462
    elif conversion_type == "lbs_to_kg":
        return value / 2.20462
    elif conversion_type == "l_to_gal":
        return value * 0.264172
    elif conversion_type == "gal_to_l":
        return value / 0.264172
    else:
        raise ValueError("Unsupported conversion type")
try:
    print("Choose Conversion Type:")
    print("1. meters → feet")
    print("2. feet → meters")
    print("3. kilograms → pounds")
    print("4. pounds → kilograms")
    print("5. liters → gallons")
    print("6. gallons → liters")

    choice = input("Enter option number: ")
    value = float(input("Enter value to convert: "))

    mapping = {
        "1": "m_to_ft",
        "2": "ft_to_m",
        "3": "kg_to_lbs",
        "4": "lbs_to_kg",
        "5": "l_to_gal",
        "6": "gal_to_l"
    }

    if choice not in mapping:
        raise ValueError("Invalid conversion option")

    result = convert_units(value, mapping[choice])

    print("Converted value:", result)

except ValueError as e:
    print("Error:", e)
```

```
Choose Conversion Type:
1. meters → feet
2. feet → meters
3. kilograms → pounds
4. pounds → kilograms
5. liters → gallons
6. gallons → liters
Enter option number: 1
Enter value to convert: 23
Converted value: 75.45932
```

```
def find_sum(numbers):
    return sum(numbers)
def find_average(numbers):
    return sum(numbers) / len(numbers)
def find_maximum(numbers):
    return max(numbers)
def find_minimum(numbers):
    return min(numbers)
try:
    print("Choose operation:")
    print("1. Sum")
    print("2. Average")
    print("3. Maximum")
    print("4. Minimum")
```

```
    choice = input("Enter option: ")

    numbers = list(map(float, input("Enter numbers separated by space: ").split()))

    if not numbers:
        raise ValueError("List cannot be empty")

    if choice == "1":
        print("Sum:", find_sum(numbers))
    elif choice == "2":
        print("Average:", find_average(numbers))
    elif choice == "3":
        print("Maximum:", find_maximum(numbers))
    elif choice == "4":
        print("Minimum:", find_minimum(numbers))
    else:
        raise ValueError("Invalid operation")

except ValueError as e:
    print("Error:", e)
```

```
Choose operation:
1. Sum
2. Average
3. Maximum
4. Minimum
Enter option: 1
Enter numbers separated by space: 2 3
Sum: 5.0
```

```
def extract_every_other(lst):
    return lst[::2]
def get_sublist(lst, start, end):
    return lst[start:end+1]
def reverse_list(lst):
    return lst[::-1]
def remove_first_last(lst):
    return lst[1:-1]
def get_first_n(lst, n):
    return lst[:n]
def get_last_n(lst, n):
    return lst[-n:]
def reverse_skip(lst):
    return lst[-2::-2]
data = [1,2,3,4,5,6]
print(extract_every_other(data))
print(get_sublist(data,2,4))
print(reverse_list(data))
print(remove_first_last(data))
print(get_first_n(data,3))
print(get_last_n(data,2))
print(reverse_skip(data))
```

```
[1, 3, 5]
[3, 4, 5]
[6, 5, 4, 3, 2, 1]
[2, 3, 4, 5]
[1, 2, 3]
[5, 6]
[5, 3, 1]
```

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result


def access_nested_element(lst, indices):
    element = lst
    for i in indices:
        element = element[i]
    return element
```

```
def sum_nested(lst):
    total = 0
    for item in lst:
        if isinstance(item, list):
            total += sum_nested(item)
        else:
            total += item
    return total


def remove_element(lst, elem):
    result = []
    for item in lst:
        if isinstance(item, list):
            cleaned = remove_element(item, elem)
            if cleaned:
                result.append(cleaned)
        else:
            if item != elem:
                result.append(item)
    return result


def find_max(lst):
    values = flatten(lst)
    return max(values)


def count_occurrences(lst, elem):
    values = flatten(lst)
    return values.count(elem)


def deep_flatten(lst):
    return flatten(lst)


def average_nested(lst):
    values = flatten(lst)
    return sum(values) / len(values)
nested = [[1,2],[3,[4,5]],6]

print(flatten(nested))
print(access_nested_element([[1,2,3],[4,5,6],[7,8,9]],[1,2]))
print(sum_nested(nested))
print(remove_element([[1,2],[3,2],[4,5]],2))
print(find_max(nested))
print(count_occurrences([[1,2],[2,3],[2,4]],2))
print(deep_flatten([[[1,2],[3,4]],[[5,6],[7,8]]]))
print(average_nested([[1,2],[3,4],[5,6]]))
```

```
[1, 2, 3, 4, 5, 6]
6
21
[[1], [3], [4, 5]]
6
3
[1, 2, 3, 4, 5, 6, 7, 8]
3.5
```

Start coding or generate with AI.

Double-click (or enter) to edit

```
arr_ones = np.ones((4, 2))
print(arr_ones)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

```
arr_full = np.full((3, 3), 7)
print(arr_full)
```

```
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

```
original = np.array([[1, 2], [3, 4]])

arr_zeros_like = np.zeros_like(original)
print(arr_zeros_like)
```

```
[[0 0]
 [0 0]]
```

```
arr_ones_like = np.ones_like(original)
print(arr_ones_like)
```

```
[[1 1]
 [1 1]]
```

```
new_list = [1, 2, 3, 4]

arr_from_list = np.array(new_list)
print(arr_from_list)
```

```
[1 2 3 4]
```

```
arr = np.arange(10, 50)
print(arr)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

```
matrix_3x3 = np.arange(9).reshape(3, 3)
print(matrix_3x3)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```
identity_matrix = np.eye(3)
print(identity_matrix)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
random_array = np.random.random(30)
mean_value = random_array.mean()

print("Array:", random_array)
print("Mean:", mean_value)
```

```
Array: [0.65121316 0.07802316 0.82366111 0.64975102 0.81356258 0.88363515
 0.36345232 0.79268324 0.93151391 0.49216754 0.06672297 0.52285329
 0.59200669 0.82155616 0.41122901 0.95602392 0.33636808 0.69898345
 0.18318153 0.35483732 0.04024951 0.30304893 0.52653794 0.28453876
 0.17435781 0.68346285 0.40179646 0.77875952 0.56837995 0.93437562]
Mean: 0.5372977653465499
```

```
random_10x10 = np.random.random((10, 10))

print("Minimum:", random_10x10.min())
print("Maximum:", random_10x10.max())
```

```
Minimum: 0.0012661818922871104
Maximum: 0.998028685372914
```

```
zero_array = np.zeros(10)
zero_array[4] = 1

print(zero_array)
```

```
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

```
arr = np.array([1, 2, 0, 0, 4, 0])

reversed_arr = arr[::-1]
print(reversed_arr)
```

```
[0 4 0 0 2 1]
```

```
border_array = np.ones((5, 5))
border_array[1:-1, 1:-1] = 0

print(border_array)
```

```
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

```
checkerboard = np.zeros((8, 8), dtype=int)

checkerboard[1::2, ::2] = 1
checkerboard[::2, 1::2] = 1

print(checkerboard)
```

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

```
import numpy as np

x = np.array([[1, 2],
              [3, 5]])

y = np.array([[5, 6],
              [7, 8]])

v = np.array([9, 10])
w = np.array([11, 12])
```

```
add_xy = x + y
print(add_xy)
```

```
[[ 6  8]
 [10 13]]
```

```
sub_xy = x - y
print(sub_xy)
```

```
[[-4 -4]
 [-4 -3]]
```

```
mul_x = x * 3
print(mul_x)
```

```
[[ 3  6]
 [ 9 15]]
```

```
square_x = x ** 2
print(square_x)
```

```
[[ 1  4]
 [ 9 25]]
```

```
dot_vw = np.dot(v, w)
print(dot_vw)
```

```
219
```

```
dot_xv = np.dot(x, v)
print(dot_xv)
```

```
[29 77]
```

```
dot_xy = np.dot(x, y)
print(dot_xy)
```

```
[[19 22]
 [50 58]]
```

```
concat_row = np.concatenate((x, y), axis=0)
print(concat_row)
```

```
[[1 2]
 [3 5]
 [5 6]
 [7 8]]
```

```
concat_col = np.concatenate((v.reshape(2,1), w.reshape(2,1)), axis=1)
print(concat_col)
```

```
[[ 9 11]
 [10 12]]
```

Start coding or generate with AI.